Planetary Rendering Toolkit – Documentation

1. Scripts

PlanetaryFirstPersonController.cs

Handles player movement and gravity on spherical planets.

Gravity is always directed toward the planet's center.

Camera is rotated relative to the player.

Parameters:

planet – Transform reference to the planet center.

playerCamera – player's main camera.

groundLayers – defines which layers count as ground.

How it works:
The script constantly recalculates the downward vector toward the planet's center and aligns the player accordingly. Movement input is handled in local axes relative to gravity.

AtmosphereController.cs

Controls the visual atmosphere layer around a planet.

Adjusts transparency and density based on camera angle and height.

Parameters:

planet – planet Transform.

atmosphereMaterial – material with the atmosphere shader.

atmosphereHeight – thickness of the atmosphere above the surface.

How it works:
An atmosphere sphere is placed around the planet. The shader uses planet radius and atmosphere height to calculate color blending and density.

PlanetChunk.cs

Generates a piece of the planet mesh.

A planet is built from 6 cube faces that are deformed into a sphere.

Parameters:

Face – selects which cube side to generate (Up, Down, Left, Right, Front, Back).

How it works:
Each chunk generates a mesh, normalizes vertex positions, and scales them to form a spherical surface. Together, all chunks form a procedural planet.

SpotlightToggleSimple.cs

Toggles a Light component on and off with a mouse click.

Parameters:

spotlight – reference to the Light (auto-assigned if attached to the same object).

How it works:
On left mouse click, the script enables or disables the Light. Useful for testing or adding interactive spotlights.

SunOrbit.cs

Moves an object (e.g., the sun) in orbit around the planet.

Parameters:

planet – orbit center (Transform).

orbitRadius – orbit distance from the planet.

orbitSpeedDeg – orbit speed in degrees per second.

initialAngleDeg – starting orbit angle.

inclinationDeg – orbit tilt relative to world Y-axis.

lookAtPlanet – if enabled, the object will always face the planet.

How it works:
Calculates orbital position analytically using rotation around world axes. Keeps the motion stable and independent of local transforms.

2. Shaders
Toon Water Shader

Creates stylized water with depth-based color and foam.

Color changes based on water depth.

Foam appears around objects using noise textures and distortion.

How it works:
Reads scene depth (_CameraDepthTexture) to detect how far objects are beneath the water surface. Blends shallow, mid, and deep colors, and applies animated foam with noise textures.

Planet Vertex Color Diffuse Shader

Combines vertex colors with a texture.

Boosts vertex colors for more vibrant terrain.

Parameters:

_MainTex – base texture.

_ColorBoost – multiplier for vertex color intensity.

How it works:
Vertex colors from the mesh are multiplied by the texture color and boosted with _ColorBoost. Great for painting terrain details directly in the mesh.

Atmosphere BRP Shader (Visible in Dark)

Adds a visible atmosphere glow around planets.

Works even in low light conditions.

Parameters:

_Color – atmosphere color and alpha.

_Intensity – overall density.

_PlanetPos – planet world position.

_PlanetRadius – planet size.

_AtmoHeight – atmosphere thickness.

_VisibilityThreshold / _VisibilitySmooth – controls visibility fade.

_DirectLightBoost – adjusts direct light contribution.

How it works:
Calculates height above the planet surface and viewing angle. Adds density near the horizon. Blends ambient and directional light to keep atmosphere visible, even when not directly lit.